ADA LAB MANUAL

V Semester BCA

1. Program to Search an Element by Sequential Search Method and Estimate Time and Space Complexity

C++ Program

```
#include <iostream>
using namespace std;
int main()
{
   int n, key, i, t, s;
   cout << "Enter number of elements: ";
   cin >> n;
   int a[n];
   cout << "Enter elements:\n";
   t = t + 2;
   for (i = 0; i < n; i++)
   {
      t = t + 2;
      cin >> a[i];
   }
   cout << "Enter element to search: ";
   cin >> key;
   s = sizeof(n) + sizeof(key) + sizeof(i)
      + sizeof(t) + sizeof(s) + sizeof(a);
   cout << "Size Complexity is " << s << "\n";
   t = t + 2;
```

```
    for (i = 0; i < n; i++)
    {
        t = t + 3;
        if (a[i] == key)
        {
            cout << "Element found at position "
                 << i + 1 << "\n";
            cout << "Time Complexity is " << t << "\n";
            return 0;
        }
    }
    cout << "Element not found\n";
    cout << "Time Complexity is " << t << "\n";
    return 0;
}
```

Output:

Output (partial in source PDF):

Enter number of elements: 5

Enter elements:

1

2

3

2. Program to Generate Fibonacci Series Without Recursion and Estimate Time and Space Complexity

C++ Program

```
#include <iostream>
```

```
using namespace std;
int main()
{
    int n, i, t, s, a = 0, b = 1, c;
    cout << "Enter number of terms: ";
    cin >> n;
    t = t + 4;
    s = sizeof(n) + sizeof(a) + sizeof(b)
        + sizeof(c) + sizeof(i) + sizeof(t)
        + sizeof(s);
    for (i = 1; i <= n; i++)
    {
        t = t + 2;
        cout << a << "\n";
        c = a + b;
        a = b;
        b = c;
        t = t + 4;
    }
    cout << "Space Complexity is " << s << "\n";
    cout << "Time Complexity is " << t << "\n";
    return 0;
}
```

Output:

Output:

Enter number of terms: 5

0

1

1

2

3

Space Complexity is 28

Time Complexity is 50

3. Program to Find Factorial of a Number Using Recursion and Estimate Time and Space Complexity

C++ Program

```
#include <iostream>
using namespace std;
int t, s;
long long fact(int n)
{
    t = t + 3;
    if (n == 0 || n == 1)
        return 1;
    t = t + 2;
    return n * fact(n - 1);
}
int main()
{
    int n;
    cout << "Enter number: ";
    cin >> n;
```

```
    s = sizeof(n);
    cout << "Factorial = " << fact(n) << "\n";
    cout << "Time Complexity is " << t << "\n";
    cout << "Space Complexity is " << s << "\n";
    return 0;
}
```

Output:

```
Output:
Enter number: 0
Factorial = 1
Time Complexity is 3
Space Complexity is 4
```

4. Program to Search an Element by Binary Search and Estimate Time and Space Complexity

C++ Program

```
#include <iostream>
using namespace std;
void display();
int t, s;
int main()
{
    int n, key, i, low, mid, high;
    cout << "Enter number of elements: ";
    cin >> n;
    int a[n];
    cout << "Enter sorted elements:\n";
```

```
t = t + 2;
for (i = 0; i < n; i++)
{
   t = t + 2;
   cin >> a[i];
}
cout << "Enter key: ";
cin >> key;
t = t + 2;
low = 0;
high = n - 1;
t = t + 1;
s = sizeof(n) + sizeof(key) + sizeof(i)
   + sizeof(low) + sizeof(mid) + sizeof(high)
   + sizeof(a);
while (low <= high)
{
   t = t + 4;
   mid = (low + high) / 2;
   t = t + 1;
   if (a[mid] == key)
   {
      t = t + 2;
      cout << "Element found at position "
          << mid + 1 << "\n";
      display();
```

```
            return 0;
        }
        else if (key < a[mid])
        {
            t = t + 2;
            high = mid - 1;
        }
        else
        {
            t = t + 2;
            low = mid + 1;
        }
    }
    cout << "Element not found\n";
    display();
    return 0;
}
void display()
{
    cout << "Time Complexity is " << t << "\n";
    cout << "Space Complexity is " << s << "\n";
}
```

Output:

```
Output:
Enter number of elements: 5
Enter sorted elements:
```

1

2

3

4

5

Enter key: 1

Element found at position 1

Time Complexity is 29

Space Complexity is 44

5. Program to Implement Merge Sort Algorithm and Estimate Time and Space Complexity

C++ Program

```
#include <iostream>

using namespace std;

int t, s;

void merge(int a[], int low, int mid, int high)

{

   t = t + 1;

   int i = low;

   t = t + 2;

   int j = mid + 1;

   t = t + 1;

   int k = low;

   int c[10];

   s = sizeof(i) + sizeof(j) + sizeof(k)

      + sizeof(c) + sizeof(a) + sizeof(low)
```

```
    + sizeof(mid) + sizeof(high);
t = t + 3;
while (i <= mid && j <= high)
{
    t = t + 3;
    if (a[i] <= a[j])
    {
        t = t + 3;
        c[k++] = a[i++];
    }
    else
    {
        t = t + 3;
        c[k++] = a[j++];
    }
}
t = t + 1;
while (i <= mid)
{
    t = t + 4;
    c[k++] = a[i++];
}
t = t + 1;
while (j <= high)
{
    t = t + 4;
```

```
        c[k++] = a[j++];
    }
    t = t + 2;
    for (i = low; i <= high; i++)
    {
        t = t + 4;
        a[i] = c[i];
    }
}
void mergeSort(int a[], int l, int r)
{
    int m;
    s = sizeof(a) + sizeof(l) + sizeof(r) + sizeof(m);
    t = t + 1;
    if (l < r)
    {
        t = t + 4;
        m = (l + r) / 2;
        mergeSort(a, l, m);
        mergeSort(a, m + 1, r);
        merge(a, l, m, r);
    }
}
int main()
{
    int n, i;
```

```
    cout << "Enter size: ";
    cin >> n;
    int a[n];
    s = sizeof(n) + sizeof(a) + sizeof(i);
    cout << "Enter unsorted elements:\n";
    t = t + 2;
    for (i = 0; i < n; i++)
    {
       t = t + 2;
       cin >> a[i];
    }
    mergeSort(a, 0, n - 1);
    cout << "Sorted array:\n";
    t = t + 2;
    for (i = 0; i < n; i++)
    {
       t = t + 2;
       cout << a[i] << " ";
    }
    cout << "\n";
    cout << "Time Complexity is " << t << "\n";
    cout << "Space Complexity is " << s << "\n";
    return 0;
}
```

Output:

Output:

Enter size: 5

Enter unsorted elements:

3

1

2

5

4

Sorted array:

12345

Time Complexity is 203

Space Complexity is 72

6. Program to Generate N Random Numbers and Sort Them Using Quick Sort

C++ Program

```
#include <iostream>
using namespace std;
int partition(int arr[], int low, int high)
{
    int pivot = arr[low];
    int i = low + 1;
    int j = high;
    while (i <= j)
    {
        // Find element greater than pivot
        while (i <= high && arr[i] <= pivot)
            i++;
```

```
        // Find element smaller than pivot
        while (j >= low && arr[j] > pivot)
            j--;
        if (i < j)
        {
            swap(arr[i], arr[j]);
        }
    }
    // Place pivot in its correct position
    swap(arr[low], arr[j]);
    return j;
}
void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int p = partition(arr, low, high);
        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}
int main()
{
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
```

```
    int arr[n];
    cout << "The array elements randomly generated are:\n";
    for (int i = 0; i < n; i++)
    {
        arr[i] = rand() % 10;
        cout << arr[i] << endl;
    }
    quickSort(arr, 0, n - 1);
    cout << "Sorted elements: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}
```

Output:

Output (sample):

Enter size: 5

The array elements randomly generated are:

1

7

4

0

9

Sorted array:

01479

7. Program to Find Minimum and Maximum Values in an Array Using Divide and Conquer

C++ Program

```
#include <iostream>
#include <climits>
using namespace std;
void MinMaxDac(int [], int, int, int &, int &);
int main()
{
    int len, max = INT_MIN, min = INT_MAX;
    cout << "Enter length of the array\n";
    cin >> len;
    int arr[len];
    cout << "Enter the elements into the array\n";
    for (int i = 0; i < len; i++)
    {
        cin >> arr[i];
    }
    MinMaxDac(arr, 0, len - 1, max, min);
    cout << "Maximum element in the array = " << max << "\n";
    cout << "Minimum element in the array = " << min << "\n";
    return 0;
}
void MinMaxDac(int arr[], int i, int j, int &max, int &min)
{
    // If array contains only one element
```

```
if (i == j)
{
    if (max < arr[i])
        max = arr[i];
    if (min > arr[i])
        min = arr[i];
    return;
}
// If array contains two elements
if (j - i == 1)
{
    if (arr[i] > arr[j])
    {
        if (max < arr[i])
            max = arr[i];
        if (min > arr[j])
            min = arr[j];
    }
    else
    {
        if (max < arr[j])
            max = arr[j];
        if (min > arr[i])
            min = arr[i];
    }
    return;
```

```
    }
    int mid = (i + j) / 2;
    // Left sub-array
    MinMaxDac(arr, i, mid, max, min);
    // Right sub-array
    MinMaxDac(arr, mid + 1, j, max, min);
}
```

Output:

Output 1:

Enter length of the array

1

Enter the elements into the array

5

Minimum element in the array = 5

Maximum element in the array = 5

Output 2:

Enter length of the array

6

Enter the elements into the array

25 10 40 5 30 15

Minimum element in the array = 5

Maximum element in the array = 40

8. Implementation of Fractional Knapsack Problem Using Greedy Approach

C++ Program

```
#include <iostream>
```

```
#include <algorithm>
using namespace std;
struct Item
{
   int value, weight;
};
bool compare(Item a, Item b)
{
   return (double)a.value / a.weight >
         (double)b.value / b.weight;
}
int main()
{
   int n, M;
   cout << "Enter number of items: ";
   cin >> n;
   Item item[n];
   cout << "Enter value and weight:\n";
   for (int i = 0; i < n; i++)
   {
      cin >> item[i].value >> item[i].weight;
   }
   cout << "Enter capacity: ";
   cin >> M;
   sort(item, item + n, compare);
   double profit = 0;
```

```
    for (int i = 0; i < n; i++)
    {
        if (M >= item[i].weight)
        {
            profit = profit + item[i].value;
            M = M - item[i].weight;
        }
        else
        {
            profit = profit +
                    (double)item[i].value * M /
                    item[i].weight;
            break;
        }
    }
    cout << "Maximum profit = " << profit;
    return 0;
}
```

Output:

Output:

Enter number of items: 5

Enter value and weight:

5 1

10 3

15 5

7 4

8 1

Enter capacity: 15

Maximum profit = 45

9. Program to Find the Minimum Cost Spanning Tree Using Prim's Algorithm and Estimate Time and Space Complexity

C++ Program

```
#include <iostream>
using namespace std;
#define V 5
int main()
{
    int t, s, minm, x, y, i, j;
    int cost = 0; // Variable to store total cost
    t = t + 25;
    int graph[V][V] =
    {
        {0, 2, 0, 6, 0},
        {2, 0, 3, 8, 5},
        {0, 3, 0, 0, 7},
        {6, 8, 0, 0, 9},
        {0, 5, 7, 9, 0}
    };
    t = t + 1;
    int selected[V] = {0};
    t = t + 1;
    selected[0] = 1;
```

```
t = t + 1;
int edges = 0;
t = t + 2;
while (edges < V - 1)
{
   t = t + 5;
   minm = 999, x = 0, y = 0;
   t = t + 2;
   for (i = 0; i < V; i++)
   {
      t = t + 3;
      if (selected[i])
      {
         t = t + 3;
         for (j = 0; j < V; j++)
         {
            t = t + 5;
            if (!selected[j] && graph[i][j])
            {
               t = t + 4;
               if (minm > graph[i][j])
               {
                  t = t + 3;
                  minm = graph[i][j];
                  x = i;
                  y = j;
```

```
                    }
                }
            }
        }
    }
    cout << x << " - " << y << " = "
        << graph[x][y] << endl;
    cost = cost + graph[x][y];
    t = t + 2;
    selected[y] = 1;
    edges++;
  }
  cout << "Total Cost of MST = " << cost << endl;
  s = sizeof(V) + sizeof(graph) + sizeof(selected)
    + sizeof(minm) + sizeof(x) + sizeof(y)
    + sizeof(i) + sizeof(j) + sizeof(cost);
  cout << "Time Complexity is: " << t << "\n";
  cout << "Space Complexity is: " << s << "\n";
  return 0;
}
```

Output:

Output:

0-1=2

1-2=3

1-4=5

0-3=6

Total Cost of MST = 16

Time Complexity is: 476

Space Complexity is: 148

10. Program to Find the Minimum Cost Spanning Tree Using Kruskal's Algorithm and Estimate Time and Space Complexity

C++ Program

```
#include <iostream>
using namespace std;
#define V 5
#define E 7
struct Edge
{
    int src, dest, weight;
};
int main()
{
    // Initializing tracking metrics
    int t = 0, s, minm, x, y, i, j;
    // Defining the graph connections
    Edge edges[E] =
    {
        {0, 1, 2},
        {0, 3, 6},
        {1, 2, 3},
        {1, 3, 8},
        {1, 4, 5},
```

```
    {2, 4, 7},
    {3, 4, 9}
};
t = t + 1;
// Disjoint set array for cycle detection
int parent[V];
t = t + 2;
for (i = 0; i < V; i++)
{
    t = t + 3;
    parent[i] = i;
}
// 1. Sorting the edges
t = t + 2;
for (i = 0; i < E - 1; i++)
{
    t = t + 3;
    for (j = i + 1; j < E; j++)
    {
        t = t + 5;
        if (edges[i].weight > edges[j].weight)
        {
            t = t + 3;
            Edge temp = edges[i];
            edges[i] = edges[j];
            edges[j] = temp;
```

```
        }
    }
}
int mst_edges_count = 0;
int total_cost = 0; // Stores minimum cost
t = t + 1;
cout << "Edges in the Minimum Spanning Tree:\n";
// 2. Kruskal's routine
int k = 0;
while (mst_edges_count < V - 1 && k < E)
{
    t = t + 5;
    Edge current_edge = edges[k];
    // Find root of source vertex
    t = t + 2;
    int root_src = current_edge.src;
    while (parent[root_src] != root_src)
    {
        t = t + 3;
        root_src = parent[root_src];
    }
    // Find root of destination vertex
    t = t + 2;
    int root_dest = current_edge.dest;
    while (parent[root_dest] != root_dest)
    {
```

```
        t = t + 3;
        root_dest = parent[root_dest];
    }
    t = t + 4;
    // If roots are different, no cycle is formed
    if (root_src != root_dest)
    {
        cout << current_edge.src << " - "
             << current_edge.dest << " = "
             << current_edge.weight << endl;
        t = t + 3;
        // Perform union
        parent[root_src] = root_dest;
        t = t + 1;
        // Add edge weight to total cost
        total_cost = total_cost + current_edge.weight;
        mst_edges_count++;
        t = t + 1;
    }
    k++;
}
// Display minimum cost
cout << "\nMinimum Cost of Spanning Tree = "
     << total_cost << endl;
// Calculating space allocation
s = sizeof(V) + sizeof(E) + sizeof(edges)
```

```
        + sizeof(parent) + sizeof(mst_edges_count)
        + sizeof(total_cost) + sizeof(k) + sizeof(minm)
        + sizeof(x) + sizeof(y) + sizeof(i) + sizeof(j);
    // Display time and space complexity counters
    cout << "\nTime Complexity Counter (t): "
         << t << endl;
    cout << "Space Complexity Counter (s): "
         << s << " bytes" << endl;
    return 0;
}
```

Output:

Output:

Edges in the Minimum Spanning Tree:

0-1=2

1-2=3

1-4=5

0-3=6

Minimum Cost of Spanning Tree = 16

Time Complexity Counter (t): 240

Space Complexity Counter (s): 144 bytes